# Brightloop: API Spec & Backend Expectations

Oct 31, 2026 · @FINSPREE SOLUTIONS

## 1. How we work

This doc is the contract between frontend and backend. If an endpoint, field or error code is not written here, it does not exist yet. Backend builds to it; frontend mocks it from hour one, so neither side waits.

| Area | Owner |
|---|---|
| Express API, MongoDB, background jobs, emails | Backend |
| Local Docker, CI/CD, AWS Elastic Beanstalk deployment, Brevo email, seed data, test accounts | Backend |
| React web app: customer site (sign-up, login, portal) and staff console (login only) | Frontend |
| KMP customer app (Android, iOS, desktop) | Frontend |
| This doc | Both: change it here first, then in code |

1. **Contract first.** Names, shapes and error codes below are final unless changed here. To change one, comment on the line and agree before coding.
2. **Frontend never waits.** Every screen is built against mock JSON copied from this doc. When an endpoint goes live, only the base URL changes.
3. **Backend never waits.** The API returns everything a screen needs in one call: related names (section 2) and the buttons allowed ( `allowedActions` ). Frontend layout choices never change the API.
4. **Done means deployed.** An endpoint counts as done when it works on the shared dev URL with seed data (section 16).
5. **Backend hands over:** the dev base URL, a Postman or Bruno collection kept in the repo, the seed script and the test accounts.

Companion files shared with this spec: `brightloop-wireframes.html` (every screen these endpoints serve) and `brightloop-user-journey.html` (the end-to-end flow). Open them in any browser.

# 2. API conventions

Every endpoint follows these rules. Both the React app and the KMP app depend on them, so treat them as non-negotiable.

| Topic | Rule |
|---|---|
| Base URL | `/api/v1`. Local: `http://localhost:4000/api/v1`. Deployed URL added here once live. |
| Format | JSON only. `Content-Type: application/json`. |
| Auth | `Authorization: Bearer <token>` header on every call except sign-up, the two logins, forgot password and reset password. No cookies (mobile can't rely on them). |
| Token | JWT with `{ sub: userId, role }`, valid 7 days. No refresh token in v1. Tokens issued before the user's last password change are rejected. A 401 means "log in again". |
| IDs | MongoDB ObjectId sent as a string field named `id`. Never expose `_id` or `__v`. |
| Human numbers | Subscriptions `SUB/2026/0042`, invoices `INV/2026/0310`. Sequential per year, never reused. |
| Money | **Whole paise as integers.** `456000` = ₹4,560.00. Never floats. Frontend formats for display. Single currency: INR. |
| Percentages | Plain numbers: `18` means 18%. Up to 2 decimals allowed (`12.5`). |
| Dates | Calendar dates as `"YYYY-MM-DD"` (startDate, dueDate). Timestamps as ISO UTC `"2026-10-31T09:30:00Z"` (createdAt). |
| Business time zone | `Asia/Kolkata`. "Today", "overdue" and "due for billing" all use the IST date. |
| Related records | Requests send IDs (`customerId`). Responses embed a small summary (`customer: { id, name }`) so lists need no extra calls. |
| Deleting | Records referenced by invoices are archived (`isActive: false`), never removed. |

| Topic | Rule |
|---|---|
| Timestamps | Every document has `createdAt` and `updatedAt`. |
| CORS | Allow origins listed in `CORS_ORIGINS`. |

## Lists

Every list endpoint is paginated on the server. Query parameters: `page` (from 1), `limit` (default 20, max 100), `sort` (field name, `-` prefix for descending, e.g. `-createdAt`), `search`, plus the filters listed per endpoint.

```
{
  "data": [ { "id": "6721f0c2a1b2c3d4e5f60718", "number": "SUB/2026/0042" }
],
  "meta": { "page": 1, "limit": 20, "total": 1412, "totalPages": 71 }
}
```

Single records come back as `{ "data": { ... } }`. Deletes return `204` with no body.

## Errors

Every error uses one shape. `details` lists field problems for forms.

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Some fields are invalid",
    "details": [ { "field": "password", "message": "Must contain a special
character" } ]
  }
}
```

| HTTP | Code | When |
|---|---|---|
| 400 | `VALIDATION_ERROR` | Body or query fails validation |
| 400 | `IMPORT_INVALID` | CSV file can't be read: wrong type, too big, missing columns, too many rows |
| 400 | `TOKEN_INVALID` | Reset link is wrong, already used or expired |
| 401 | `UNAUTHENTICATED` | Missing, bad or expired JWT |
| 401 | `INVALID_CREDENTIALS` | Wrong email or password, or the account belongs on the other login page (same message for all) |

| HTTP | Code | When |
|---|---|---|
| 403 | `FORBIDDEN` | Logged in, but this role can't do this |
| 403 | `ACCOUNT_DISABLED` | User was disabled by an admin |
| 403 | `PASSWORD_CHANGE_REQUIRED` | User still has a temporary password and must change it first |
| 404 | `NOT_FOUND` | Record missing, or belongs to another customer |
| 409 | `EMAIL_TAKEN` | Email already has an account |
| 409 | `INVALID_TRANSITION` | Action not allowed in the current status |
| 409 | `NOT_EDITABLE` | Record is locked in its current status |
| 409 | `ALREADY_INVOICED` | This subscription period already has an invoice |
| 409 | `OVERPAYMENT` | Payment is larger than the invoice balance |
| 409 | `HAS_PAYMENTS` | Can't cancel an invoice that has payments |
| 409 | `PLAN_RULE` | Plan doesn't allow this (pause, renew, close early) |
| 409 | `BELOW_MIN_QUANTITY` | Quantity under the plan's minimum |
| 409 | `PLAN_NOT_AVAILABLE` | Start date is outside the plan's available dates |
| 409 | `LAST_ADMIN` | Would leave the system with no active admin |
| 429 | `RATE_LIMITED` | Too many attempts (login, forgot password, import) |
| 500 | `INTERNAL` | Anything unexpected. Log it with a request ID. |

## 3. Roles and permissions

Four roles: `admin`, `staff`, `finance`, `customer`. Check the role on the server for every route; the frontend hiding a button is not security.

| Resource | Admin | Staff | Finance | Customer |
|---|---|---|---|---|
| Login page | Staff console `/staff/login` | Staff console | Staff console | Customer site `/login` |
| Sign up | Never | Never | Never | Yes, on the customer site |

| Resource | Admin | Staff | Finance | Customer |
|---|---|---|---|---|
| Forgot password, change password, own profile | Yes | Yes | Yes | Yes |
| Users: create one, bulk import (CSV), disable, reset password | Yes | No | No | No |
| Products, plans | Full | Read | Read | None |
| Quotation templates | Full | Full | Read | None |
| Taxes, discounts | Full | Read | Read | None |
| Customers | Full (archive, no delete) | Full (archive, no delete) | Read | Own record via `/auth/me` |
| Subscriptions | Full | Full | Read | Own, via `/portal` |
| Invoices: confirm, cancel, edit draft | Yes | Yes | No | No |
| Invoices: read, send or resend | Yes | Yes | Yes | Own, read only, never drafts |
| Billing run | Run + preview | Run + preview | Preview only (`dryRun`) | No |
| Payments | Record, read, delete | Record, read | Record, read | Own, read only (on invoice) |
| Dashboard, reports, CSV export | Yes | Yes | Yes | No (portal summary only) |

Rules that come straight from the brief:

- **Staff accounts can never sign up.** Admin, internal staff and finance lead accounts are created only by an admin: one at a time or by CSV import (section 7). Sign-up always creates a `customer`; ignore any `role` in its body.

- **Each login page accepts only its own people.** The customer site rejects staff accounts and the staff console rejects customers, with the same error as a wrong password (section 5).
- **CSV access levels map to roles:** Internal Staff → `staff`, Finance Lead → `finance`, Admin → `admin`.
- **Only an admin can create discount rules.**
- **Customers see only their own subscriptions and invoices.** Every `/portal` query filters by the logged-in user's `customerId`. Never accept a `customerId` from a customer's request.
- Another customer's record returns `404`, not `403`, so IDs can't be probed.
- **Finance is the finance lead's role.** It reads everything, records payments, sends invoice reminders and owns the reports. It can't change subscriptions, the catalog or an invoice's status.
- Use one middleware, `requireRole("admin", "staff", ...)`, on every route, matching the Who column of the endpoint tables below.

## 4. Data models (MongoDB)

Ten collections. Lines and variants are embedded; everything else is linked by ID. Invoices copy (snapshot) names, prices and rates, so later edits never change an old invoice.

Storage rules for every collection:

- Money: integer paise. Calendar dates: `"YYYY-MM-DD"` strings (they sort correctly and have no time-zone bugs). Timestamps: `Date`.
- Mongoose `timestamps: true`, and a `toJSON` transform that renames `_id` to `id` and drops `__v` and secrets.
- Status fields are Mongoose enums, so bad values can't be saved.

### users

Login accounts for all four roles. Staff, finance and admin users are created by an admin (one at a time or by CSV) with a temporary password; customers create themselves through sign-up.

```
users {
  name:                String, required
  email:               String, required, lowercase, trimmed   // UNIQUE
  passwordHash:        String, required                       // bcrypt,
never returned
  role:                "admin" | "staff" | "finance" | "customer"
  status:              "active" | "disabled"
  mustChangePassword:  Boolean, default false   // true for admin-created and
CSV-imported users
  passwordChangedAt:   Date | null              // JWTs issued before this
are rejected
  createdVia:          "signup" | "admin" | "import" | "seed"
  createdBy:           ObjectId -> users | null // the admin who created or
imported them
  customerId:          ObjectId -> customers | null           // set only for
role "customer"
  lastLoginAt:         Date | null
  resetTokenHash:      String | null            // sha256 of the forgot-
password token
  resetTokenExpiresAt: Date | null
}
// indexes: { email: 1 } unique, { role: 1, status: 1 }
```

## customers

The business being billed. Staff can create a customer without a login; when someone later signs up with that email, the new login links to it.

```
customers {
  name:           String, required          // company or person
  contactName:    String
  email:          String, required, lowercase
  phone:          String
  billingAddress: { line1, line2, city, state, postalCode, country }
  taxId:          String                    // GSTIN
  notes:          String
  userId:         ObjectId -> users | null  // portal login, if any
  isActive:       Boolean, default true
}
// indexes: { email: 1 }, text index on { name, email } for search
```

## products

```
products {
  name:       String, required
  type:       "service" | "addon" | "physical"
  salesPrice: Int (paise)
  costPrice:  Int (paise)
  notes:      String
  variants: [ {                     // embedded; each has its own _id
    attribute:  String              // "Brand"
    value:      String              // "Odoo"
    extraPrice: Int (paise)         // 56000 = +₹560
  } ]
  isActive:   Boolean, default true
}
// indexes: { name: 1 }, { isActive: 1, type: 1 }
```

## plans

How a product is sold on repeat: price per unit per period, minimum quantity, availability and rules.

```
plans {
  name:           String, required
  productId:      ObjectId -> products     // the main product this plan
sells
  price:          Int (paise)             // per unit, per billing period
  billingPeriod:  "daily" | "weekly" | "monthly" | "yearly"
  minQuantity:    Int >= 1
  availableFrom:  "YYYY-MM-DD"
  availableUntil: "YYYY-MM-DD" | null      // null = no end
  rules: {
    autoClose: Boolean   // close automatically on the expiry date
    closable:  Boolean   // can be closed early
    pausable:  Boolean
    renewable: Boolean
  }
  description:    String
  isActive:       Boolean, default true
}
// indexes: { isActive: 1, availableFrom: 1 }
```

## quotationTemplates

```
quotationTemplates {
  name:            String, required
  planId:          ObjectId -> plans
  validityDays:    Int        // quotation stays valid N days after it is
sent
  paymentTermDays: 0 | 15 | 30 | 45
  lines: [ { productId, variantId | null, quantity } ]
  notes:           String     // terms printed on the quotation
  isActive:        Boolean, default true
}
```

## taxes

```
taxes {
  name:  String              // "GST 18%"
  type:  "percentage" | "fixed"
  rate:  Number              // percentage: 18 means 18%. fixed: paise per
unit
  appliesTo: {
    kind:         "all" | "productTypes" | "products"
    productTypes: ["service" | "addon" | "physical"]   // when kind =
productTypes
    productIds:   [ObjectId]                          // when kind = products
  }
  isActive: Boolean, default true
}
```

## discounts

```
discounts {
  name:              String
  type:              "percentage" | "fixed"
  value:             Number           // percentage: 10 means 10%. fixed:
paise
  minPurchaseAmount: Int | null       // paise, compared with the
subscription subtotal
  minQuantity:       Int | null
  startDate:         "YYYY-MM-DD"
  endDate:           "YYYY-MM-DD"
  usageLimit:        Int | null       // null = unlimited
  usedCount:         Int, default 0   // 1 use = 1 invoice it was applied to
  appliesTo: {
    kind:       "products" | "subscription"   // specific products, or the
whole subscription
    productIds: [ObjectId]                    // when kind = products
  }
  isActive:          Boolean, default true
}
// indexes: { isActive: 1, startDate: 1, endDate: 1 }
```

## subscriptions

```
subscriptions {
  number:              String           // "SUB/2026/0042", UNIQUE
  customerId:          ObjectId -> customers
  planId:              ObjectId -> plans
  templateId:          ObjectId -> quotationTemplates | null
  salespersonId:       ObjectId -> users | null
  status:              "draft" | "quotation" | "confirmed" | "active" |
"paused" | "closed"
  startDate:           "YYYY-MM-DD"
  expiryDate:          "YYYY-MM-DD"     // default: startDate + 1 year - 1
day
  quotationValidUntil: "YYYY-MM-DD" | null
  paymentTermDays:     Int
  lines: [ {                            // embedded; each has its own _id
    productId, productName,             // name copied at creation
    variantId | null, variantLabel,     // "Brand: Odoo"
    quantity:  Int
    unitPrice: Int (paise)              // price copied at creation (section
14)
  } ]
  nextInvoiceDate:     "YYYY-MM-DD" | null   // first day of the next period
```

```
to bill
  recurringTotal:      Int (paise)            // one period's total incl.
tax; refresh when lines change
  recurringNet:        Int (paise)            // same, after discounts and
before tax (used for MRR)
  pausedAt, closedAt:  Date | null
  closeReason:         "expired" | "closed_early" | "cancelled" | "rejected"
| "quotation_expired" | null
  notes:               String
  history: [ { at: Date, by: userId | "system", action, from, to, note } ]
}
// indexes: { number: 1 } unique, { status: 1, nextInvoiceDate: 1 },
//          { customerId: 1, status: 1 }, { planId: 1, status: 1 }
```

## invoices

```
invoices {
  number:           String           // "INV/2026/0310", UNIQUE
  subscriptionId:   ObjectId -> subscriptions
  customerId:       ObjectId -> customers
  customerSnapshot: { name, email, billingAddress, taxId }
  periodStart:      "YYYY-MM-DD"
  periodEnd:        "YYYY-MM-DD"
  invoiceDate:      "YYYY-MM-DD"
  dueDate:          "YYYY-MM-DD"     // invoiceDate + paymentTermDays
  status:           "draft" | "confirmed" | "paid" | "cancelled"
  isVoid:           Boolean, default false   // true only when cancelled
  lines: [ {
    productId, description,          // "Brightloop CRM · Brand: Odoo"
    quantity, unitPrice,
    subtotal,                        // quantity x unitPrice
    discount: { discountId, name, amount } | null,
    orderDiscountShare: Int,         // this line's share of a whole-
subscription discount
    taxes: [ { taxId, name, type, rate, amount } ],
    taxAmount, total
  } ]
  orderDiscount:      { discountId, name, amount } | null
  untaxedAmount, discountTotal, taxTotal, total      // all Int paise
  amountPaid:         Int, default 0
  balance:            Int                          // total - amountPaid,
stored for fast lists
  appliedDiscountIds: [ObjectId]     // usage reserved; released on cancel
  sentAt: Date | null, sentCount: Int
  confirmedAt, paidAt, cancelledAt:  Date | null
  createdBy:          userId | "system"
```

```
}
// indexes:
//   { number: 1 } unique
//   { subscriptionId: 1, periodStart: 1 } unique, partialFilterExpression: {
isVoid: false }
//      -> the database itself refuses a second invoice for the same period
//   { status: 1, dueDate: 1 }, { customerId: 1, status: 1 }, { invoiceDate:
1 }
```

### payments

```
payments {
  invoiceId:      ObjectId -> invoices
  customerId:     ObjectId -> customers       // copied from the invoice
  subscriptionId: ObjectId -> subscriptions   // copied from the invoice
  method:         "bank_transfer" | "card" | "cash" | "cheque" | "upi" |
"other"
  amount:         Int (paise) > 0
  paymentDate:    "YYYY-MM-DD"
  note:           String
  recordedBy:     ObjectId -> users
}
// indexes: { invoiceId: 1 }, { paymentDate: -1 }, { customerId: 1 }
```

### counters

Gives each new subscription or invoice the next number, safely even when two are created at the same moment.

```
counters { _id: "SUB-2026", seq: 42 }
// next: findOneAndUpdate({ _id: "SUB-2026" }, { $inc: { seq: 1 } }, {
upsert: true, new: true })
// format: `SUB/2026/${String(seq).padStart(4, "0")}`
```

## 5. Auth and login

Two login pages share one account system. Customers sign up and log in on the customer site. Admin, internal staff and the finance lead log in on the staff console and can never sign up. Forgot password works on both pages and sends its email through Brevo.

## The two login pages

| Page (frontend route) | Used by | Calls |
|---|---|---|
| Customer site `/login` | Customers | `POST /auth/customer/login` |
| Customer site `/signup` | New customers | `POST /auth/customer/signup` |
| Customer site `/forgot-password`, `/reset-password` | Customers | `POST /auth/forgot-password` with `portal: "customer"`, then `POST /auth/reset-password` |
| Staff console `/staff/login` | Admin, internal staff, finance lead | `POST /auth/staff/login` |
| Staff console `/staff/forgot-password`, `/staff/reset-password` | Admin, internal staff, finance lead | `POST /auth/forgot-password` with `portal: "staff"`, then `POST /auth/reset-password` |

The backend needs these routes only to build the link in the reset email. There is no staff sign-up page and no staff sign-up endpoint.

## Endpoints

| Method | Path | Who | Does |
|---|---|---|---|
| POST | `/auth/customer/signup` | Public | Create a customer account, return token |
| POST | `/auth/customer/login` | Public | Log in a customer |
| POST | `/auth/staff/login` | Public | Log in an admin, staff or finance user |
| GET | `/auth/me` | Any logged in | Current user (+ customer record for customers) |
| PATCH | `/auth/me` | Any logged in | Update own name (customers: contact fields too) |
| POST | `/auth/change-password` | Any logged in | Change own password; ends a forced change |
| POST | `/auth/forgot-password` | Public | Email a reset link through Brevo |
| POST | `/auth/reset-password` | Public | Set a new password from the link |
| GET | `/meta` | Any logged in | Dropdown values and company details |

| Method | Path | Who | Does |
|---|---|---|---|
| GET | `/health` | Public | Health check for Elastic Beanstalk |

## Password rule

More than 8 characters (so 9 or more), at least one uppercase, one lowercase and one special character. Check it on sign-up, change, reset, admin-created temporary passwords and every CSV row. Return one `details` entry per rule broken, so the form can show each.

```
/^(?=.*[a-z])(?=.*[A-Z])(?=.*[^A-Za-z0-9]).{9,}$/
```

## POST /auth/customer/signup

```
// request
{ "name": "Asha Rao", "companyName": "Kite Stores", "email":
"Asha@Kite.test", "password": "Kite@2026!" }

// 201
{
  "data": {
    "token": "eyJhbGciOi...",
    "user": {
      "id": "6721f0c2a1b2c3d4e5f60701",
      "name": "Asha Rao",
      "email": "asha@kite.test",
      "role": "customer",
      "status": "active",
      "mustChangePassword": false,
      "customerId": "6721f0c2a1b2c3d4e5f60702"
    }
  }
}
```

- Email is stored lowercase. An email already used by any account (customer or staff) returns `409 EMAIL_TAKEN`.
- Always creates role `customer`. Ignore any `role` in the body.
- Also creates the `customers` record (`name` = companyName, or the person's name if empty). If staff already created a customer with this email and no login, link to it instead of making a second one.

## Logging in

```
// POST /auth/staff/login  (customer login has the same request and response)
{ "email": "meera@brightloop.test", "password": "Welcome@2026!" }

// 200
{
  "data": {
    "token": "eyJhbGciOi...",
    "user": { "id": "6721f0c2a1b2c3d4e5f60745", "name": "Meera Nair",
"email": "meera@brightloop.test", "role": "finance", "status": "active",
"mustChangePassword": true, "customerId": null }
  }
}
```

- `/auth/customer/login` accepts only role `customer`. `/auth/staff/login` accepts only `admin`, `staff` and `finance`.
- An account used on the wrong page gets `401 INVALID_CREDENTIALS`, the same message as a wrong password, so the pages don't reveal which emails exist.
- Disabled user: `403 ACCOUNT_DISABLED`, only after the password checks out.
- Sets `lastLoginAt`. Rate limit: 10 attempts per minute per IP on every `/auth` route.
- The JWT holds `{ sub, role }`. The auth middleware loads the user on every request and rejects the token if the user is disabled or the token was issued before `passwordChangedAt`.

## Forced password change (temporary passwords)

Users created by an admin or by CSV import start with a temporary password and `mustChangePassword: true`.

- Login works and returns a token with `mustChangePassword: true`. The frontend sends them straight to a "Set your password" screen.
- Until they change it, every endpoint except `GET /auth/me` and `POST /auth/change-password` returns `403 PASSWORD_CHANGE_REQUIRED`.
- `POST /auth/change-password` with `{ "currentPassword": "...", "newPassword": "..." }`. The new password must pass the rule and differ from the current one. Sets `mustChangePassword: false` and `passwordChangedAt`, and returns a fresh `{ token, user }` (the old token stops working).
- A wrong current password returns `400 VALIDATION_ERROR` on field `currentPassword`.

## POST /auth/forgot-password (Brevo)

```
// request
{ "email": "meera@brightloop.test", "portal": "staff" }

// 200, always
{ "data": { "ok": true } }
```

1. Find the user. Send nothing (still `200`) if there is no user, the user is disabled, or the role doesn't match `portal` (`customer` ↔ customer; `staff` ↔ admin, staff, finance).
2. Make a random 32-byte token. Store only its sha256 in `resetTokenHash`, with `resetTokenExpiresAt` = now + 30 minutes. A new request replaces any older token.
3. Build the link: customers get `${WEB_URL}/reset-password?token=<token>`, staff get `${WEB_URL}/staff/reset-password?token=<token>`.
4. Send it with Brevo's transactional email API:

```
POST https://api.brevo.com/v3/smtp/email
api-key: <BREVO_API_KEY>
content-type: application/json

{
  "sender": { "name": "Brightloop", "email": "<BREVO_SENDER_EMAIL>" },
  "to": [ { "email": "meera@brightloop.test", "name": "Meera Nair" } ],
  "templateId": 3,
  "params": { "name": "Meera Nair", "resetUrl": "https://<web-
url>/staff/reset-password?token=...", "expiresInMinutes": 30 }
}
```

5. Brevo answers `201 { "messageId": "..." }`. Log the messageId, never the link. If `BREVO_TEMPLATE_RESET_ID` is empty, send `subject` + `htmlContent` built in code instead of `templateId`.
6. A Brevo failure is logged and does not change the `200` response, so the page can't be used to test which emails exist.

- Limit: 3 requests per email per hour, on top of the IP limit.
- Development: `EMAIL_PROVIDER=console` logs the link instead of calling Brevo, and the response adds `"devResetUrl"` (only when `NODE_ENV=development`).
- Keep all email in one mailer module (`sendPasswordReset`, `sendQuotation`, `sendInvoice`), so switching between console and Brevo is one env var.

## POST /auth/reset-password

```
{ "token": "9f2c...", "newPassword": "New@Pass2026" }
```

- Works once: clear the token after success. Bad, used or expired token: `400 TOKEN_INVALID`.
- Sets the password, `mustChangePassword: false` and `passwordChangedAt`, so every old token for that user stops working.
- Returns `{ token, user }`. The frontend uses `user.role` to open the right app.

### GET /auth/me

Returns the user. For customers it also embeds the customer record, so the profile screen needs one call.


```
{
  "data": {
    "id": "6721f0c2a1b2c3d4e5f60701", "name": "Asha Rao", "email":
"asha@kite.test",
    "role": "customer", "status": "active", "mustChangePassword": false,
"lastLoginAt": "2026-10-31T09:30:00Z",
    "customer": { "id": "6721f0c2a1b2c3d4e5f60702", "name": "Kite Stores",
"phone": "", "billingAddress": { "city": "Pune" } }
  }
}
```


### GET /meta

One call that fills every dropdown, so neither app hard-codes lists.


```
{
  "data": {
    "accessLevels": [ { "value": "staff", "label": "Internal Staff" }, {
"value": "finance", "label": "Finance Lead" }, { "value": "admin", "label":
"Admin" } ],
    "billingPeriods": ["daily", "weekly", "monthly", "yearly"],
    "productTypes": ["service", "addon", "physical"],
    "paymentMethods": ["bank_transfer", "card", "cash", "cheque", "upi",
"other"],
    "paymentTermDays": [0, 15, 30, 45],
    "subscriptionStatuses": ["draft", "quotation", "confirmed", "active",
"paused", "closed"],
    "invoiceStatuses": ["draft", "confirmed", "paid", "cancelled"],
    "currency": "INR",
    "company": { "name": "Brightloop Software", "address": "...", "taxId":
"...", "email": "billing@brightloop.test" }
  }
}
```

### GET /health

`{ "status": "ok", "db": "ok", "time": "2026-10-31T09:30:00Z" }`, or `503` if the database is unreachable. Elastic Beanstalk polls it as `/api/v1/health`; it must answer without auth in under a second.

## 6. Catalog endpoints

Products, plans and quotation templates all follow the same five routes. Writes are admin-only, except templates, which staff can also manage.

| Method | Path | Who | Does |
|---|---|---|---|
| GET | `/products` | Admin, Staff, Finance | List. Filters: `search`, `type`, `isActive` |
| POST | `/products` | Admin | Create with variants |
| GET | `/products/:id` | Admin, Staff, Finance | One product |
| PATCH | `/products/:id` | Admin | Update; `variants` array replaces the old one |
| DELETE | `/products/:id` | Admin | Archive (`isActive: false`) |
| GET | `/plans` | Admin, Staff, Finance | List. Filters: `search`, `billingPeriod`, `isActive`, `availableOn=YYYY-MM-DD` |
| POST | `/plans` | Admin | Create |
| GET | `/plans/:id` | Admin, Staff, Finance | One plan + product summary + active subscription count |
| PATCH | `/plans/:id` | Admin | Update |
| DELETE | `/plans/:id` | Admin | Archive |
| GET | `/quotation-templates` | Admin, Staff, Finance | List. Filters: `search`, `planId`, `isActive` |
| POST | `/quotation-templates` | Admin, Staff | Create |
| GET | `/quotation-templates/:id` | Admin, Staff, Finance | One template with plan and line details |

| Method | Path | Who | Does |
|---|---|---|---|
| PATCH | `/quotation-templates/:id` | Admin, Staff | Update |
| DELETE | `/quotation-templates/:id` | Admin, Staff | Archive |

## Product

```
// POST /products
{
  "name": "Brightloop CRM",
  "type": "service",
  "salesPrice": 400000,
  "costPrice": 90000,
  "notes": "Core CRM seat",
  "variants": [
    { "attribute": "Brand", "value": "Odoo", "extraPrice": 56000 },
    { "attribute": "Brand", "value": "Generic", "extraPrice": 0 }
  ]
}

// 201, and the shape of every product in GET responses
{
  "data": {
    "id": "6721f0c2a1b2c3d4e5f60710",
    "name": "Brightloop CRM", "type": "service",
    "salesPrice": 400000, "costPrice": 90000, "notes": "Core CRM seat",
    "variants": [
      { "id": "6721f0c2a1b2c3d4e5f60711", "attribute": "Brand", "value":
"Odoo", "extraPrice": 56000 },
      { "id": "6721f0c2a1b2c3d4e5f60712", "attribute": "Brand", "value":
"Generic", "extraPrice": 0 }
    ],
    "usedInPlans": 3,
    "isActive": true,
    "createdAt": "2026-10-01T08:00:00Z", "updatedAt": "2026-10-01T08:00:00Z"
  }
}
```

- Prices are integers ≥ 0. `name` required. `type` must be one of the enum values.
- On PATCH, a variant sent with its `id` keeps that id (subscriptions point to it). A variant left out is removed only if no active subscription uses it; otherwise `409 NOT_EDITABLE`.

## Plan

```
// POST /plans
{
  "name": "Pro Yearly",
  "productId": "6721f0c2a1b2c3d4e5f60710",
  "price": 400000,
  "billingPeriod": "yearly",
  "minQuantity": 10,
  "availableFrom": "2026-01-01",
  "availableUntil": "2026-12-31",
  "rules": { "autoClose": true, "closable": false, "pausable": false,
"renewable": true },
  "description": "Yearly contract for teams of 10+"
}

// GET /plans/:id
{
  "data": {
    "id": "6721f0c2a1b2c3d4e5f60720",
    "name": "Pro Yearly",
    "product": { "id": "6721f0c2a1b2c3d4e5f60710", "name": "Brightloop CRM",
"variants": [ "...same as product..." ] },
    "price": 400000, "billingPeriod": "yearly", "minQuantity": 10,
    "availableFrom": "2026-01-01", "availableUntil": "2026-12-31",
    "rules": { "autoClose": true, "closable": false, "pausable": false,
"renewable": true },
    "activeSubscriptions": 236,
    "isActive": true
  }
}
```

- `availableUntil` must be on or after `availableFrom`. `minQuantity` ⩾ 1.
- Changing a plan's price does not change existing subscriptions. Their lines keep the price copied when they were created.

### Quotation template

```
// POST /quotation-templates
{
  "name": "Pro Yearly – Standard",
  "planId": "6721f0c2a1b2c3d4e5f60720",
  "validityDays": 14,
  "paymentTermDays": 30,
  "lines": [
    { "productId": "6721f0c2a1b2c3d4e5f60710", "variantId":
"6721f0c2a1b2c3d4e5f60711", "quantity": 10 },
    { "productId": "6721f0c2a1b2c3d4e5f60730", "variantId": null, "quantity":
1 }
  ],
  "notes": "Prices exclude GST. Billed yearly in advance."
}
```

- GET responses expand each line with `productName`, `variantLabel` and the computed `unitPrice`, plus `plan: { id, name, billingPeriod }`, and add `usedCount` (subscriptions created from it).
- The line for the plan's own product must meet the plan's `minQuantity`, otherwise `409 BELOW_MIN_QUANTITY`.

## 7. Admin config endpoints

Taxes, discounts and user accounts are admin territory. Staff and finance may read taxes and discounts (to explain a total), never change them.

| Method | Path | Who | Does |
|---|---|---|---|
| GET | `/taxes` | Admin, Staff, Finance | List. Filters: `isActive` |
| POST | `/taxes` | Admin | Create |
| PATCH | `/taxes/:id` | Admin | Update |
| DELETE | `/taxes/:id` | Admin | Archive |
| GET | `/discounts` | Admin, Staff, Finance | List. Filters: `state`, `search` |
| POST | `/discounts` | Admin | Create |
| GET | `/discounts/:id` | Admin, Staff, Finance | One discount + usage |

| Method | Path | Who | Does |
|---|---|---|---|
| PATCH | `/discounts/:id` | Admin | Update |
| POST | `/discounts/:id/stop` | Admin | Stop now (`isActive: false`) |
| GET | `/users` | Admin | List. Filters: `role`, `status`, `search` |
| POST | `/users` | Admin | Create one staff, finance or admin user with a temporary password |
| POST | `/users/import` | Admin | Bulk create from a CSV file (preview with `dryRun`) |
| GET | `/users/import/template` | Admin | Download a sample CSV |
| PATCH | `/users/:id` | Admin | Change name, role, status |
| POST | `/users/:id/reset-password` | Admin | Give a user a new temporary password |

## Tax

```
// POST /taxes
{
  "name": "GST 18%",
  "type": "percentage",
  "rate": 18,
  "appliesTo": { "kind": "productTypes", "productTypes": ["service",
"addon"], "productIds": [] },
  "isActive": true
}
```

- Which taxes apply to a line is decided by `appliesTo` (section 14). Several matching taxes all apply, each on the same base (no tax on tax).
- Editing a tax affects only invoices created afterwards.

## Discount

```
// POST /discounts
{
  "name": "Bulk seats 10%",
  "type": "percentage",
  "value": 10,
  "minPurchaseAmount": null,
  "minQuantity": 20,
  "startDate": "2026-10-01",
  "endDate": "2026-12-31",
  "usageLimit": 50,
  "appliesTo": { "kind": "products", "productIds":
["6721f0c2a1b2c3d4e5f60710"] }
}

// GET responses add:
{ "usedCount": 12, "remainingUses": 38, "state": "running" }
```

- `state` is computed, never stored: `inactive` (stopped), `scheduled` (before startDate), `running`, `expired` (after endDate), `exhausted` (usedCount = usageLimit). The `state` filter uses the same values.
- `endDate` ⩾ `startDate`. Percentage `value` between 0 and 100. `usageLimit` can't be set below `usedCount`.
- `usedCount` changes only through invoices (section 14), never through PATCH.

## Users and staff

```
// POST /users
{ "name": "Riya Sharma", "email": "riya@brightloop.test", "role": "staff",
"temporaryPassword": "Welcome@2026!" }

// 201
{
  "data": {
    "id": "6721f0c2a1b2c3d4e5f60740", "name": "Riya Sharma", "email":
"riya@brightloop.test",
    "role": "staff", "status": "active", "mustChangePassword": true,
"createdVia": "admin", "lastLoginAt": null
  }
}
```

- `role` is `staff`, `finance` or `admin`. Customers are never created here; they sign up on the customer site.

- `temporaryPassword` must pass the password rule. Hash it with bcrypt; never store, log or return it. The admin shares it with the person directly (no welcome email in v1).
- The user must change it at first login (section 5).
- PATCH accepts `name`, `role`, `status` (`active` / `disabled`). A disabled user's token stops working on the next request.
- `POST /users/:id/reset-password` with `{ "temporaryPassword": "..." }` sets a new temporary password, `mustChangePassword: true` and `passwordChangedAt`, which logs them out everywhere.
- `409 LAST_ADMIN` if a change would leave no active admin (including an admin disabling or demoting themselves).
- `GET /users?role=customer` lists customer logins, so an admin can disable one.

## Bulk import (CSV)

An admin uploads one CSV file to create many staff, finance and admin users at once. Each row gets the same checks as a single user, and the file is created whole or not at all.

```
name,email,temporary_password,access_level
Riya Sharma,riya@brightloop.test,Welcome@2026!,Internal Staff
Meera Nair,meera@brightloop.test,Welcome@2026!,Finance Lead
Arjun Mehta,arjun@brightloop.test,Welcome@2026!,Admin
```

| Column | Rule |
|---|---|
| `name` | Required, 2 to 80 characters |
| `email` | Valid email. Not used by any existing account. Not repeated in the file (case-insensitive). |
| `temporary_password` | Meets the password rule |
| `access_level` | `Internal Staff` or `staff` → staff · `Finance Lead` or `finance` → finance · `Admin` or `admin` → admin. Case-insensitive. Anything else, including `customer`, is an error. |

**Request:** `POST /users/import?dryRun=true` (or `false`), `multipart/form-data` with the file in field `file`.

- Limits: `.csv` only, at most 1 MB and 200 data rows. Anything wrong with the file itself returns `400 IMPORT_INVALID`.
- Header names are case-insensitive and trimmed. Accept UTF-8 with or without a BOM, quoted values and blank lines. Parse with `csv-parse`, never `split(",")`.

- Rate limit: 5 imports per minute per admin.

**Response** (always `200` for a preview or a rejected file; `201` when users were created):

```
{
  "data": {
    "dryRun": true,
    "total": 3, "valid": 2, "invalid": 1, "created": 0,
    "rows": [
      { "row": 2, "name": "Riya Sharma", "email": "riya@brightloop.test",
"role": "staff", "status": "valid", "errors": [] },
      { "row": 3, "name": "Meera Nair", "email": "meera@brightloop.test",
"role": "finance", "status": "valid", "errors": [] },
      { "row": 4, "name": "Arjun Mehta", "email": "arjun@brightloop", "role":
null, "status": "invalid",
        "errors": [ { "field": "email", "message": "Not a valid email" }, {
"field": "access_level", "message": "Must be Internal Staff, Finance Lead or
Admin" } ] }
    ]
  }
}
```

- `row` is the line number in the file. The header is line 1, so the first person is row 2, matching what the admin sees in a spreadsheet.
- **All or nothing.** If any row is invalid, nothing is created ( `created: 0` ). If every row is valid and `dryRun` is false, insert them all in one transaction and set each row's status to `created` .
- Imported users get `mustChangePassword: true` , `createdVia: "import"` and `createdBy` = the admin.
- Hash passwords with async bcrypt in parallel (200 rows must finish well under nginx's 60-second timeout). Never echo, log or store a password or the raw file.
- `GET /users/import/template` returns the header row plus one example row as `text/csv` .
- Frontend flow: choose file → preview ( `dryRun=true` ) → show the rows and errors → "Create N users" sends the same file with `dryRun=false` .

## 8. Customers endpoints

Staff create and manage customers; a customer's subscriptions, invoices and payments come from the main endpoints filtered by `customerId` .

| Method | Path | Who | Does |
|---|---|---|---|
| GET | `/customers` | Admin, Staff, Finance | List. Filters: `search`, `hasOverdue=true`, `hasActiveSubscription=true\|false`, `isActive` |
| POST | `/customers` | Admin, Staff | Create (no login is created) |
| GET | `/customers/:id` | Admin, Staff, Finance | One customer + stats |
| PATCH | `/customers/:id` | Admin, Staff | Update |
| DELETE | `/customers/:id` | Admin, Staff | Archive; `409 NOT_EDITABLE` if they have active subscriptions |

```
// POST /customers
{
  "name": "Acme Ltd",
  "contactName": "Vikram Iyer",
  "email": "billing@acme.com",
  "phone": "+91 98450 00000",
  "billingAddress": { "line1": "12 MG Road", "line2": "", "city":
"Bengaluru", "state": "Karnataka", "postalCode": "560001", "country": "IN" },
  "taxId": "29ABCDE1234F1Z5",
  "notes": ""
}

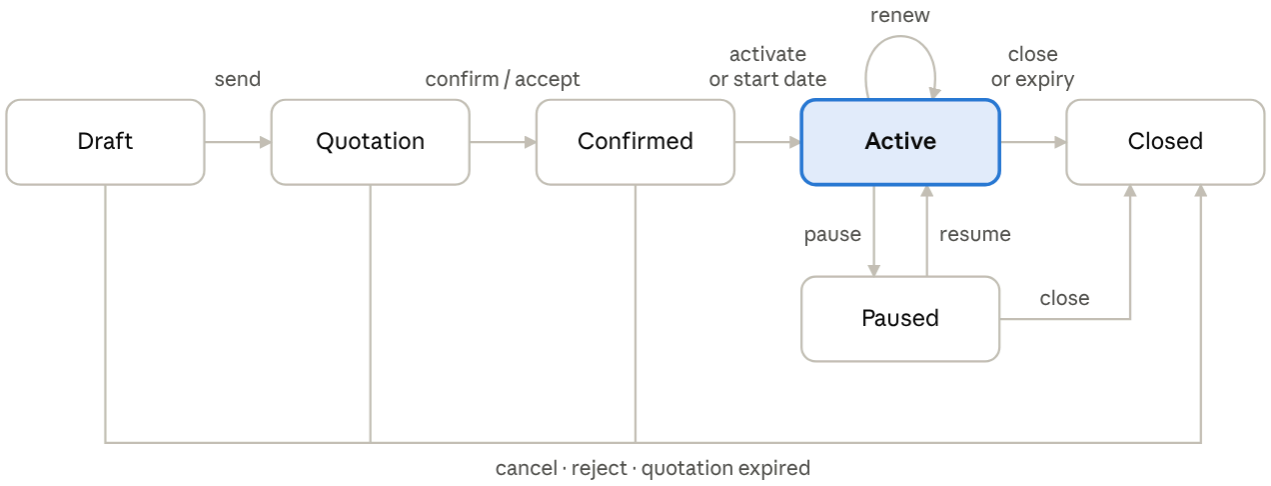
// list item and GET /customers/:id
{
  "id": "6721f0c2a1b2c3d4e5f60750",
  "name": "Acme Ltd", "contactName": "Vikram Iyer", "email":
"billing@acme.com", "phone": "+91 98450 00000",
  "billingAddress": { "...": "..." }, "taxId": "29ABCDE1234F1Z5",
  "portalStatus": "active",
  "stats": { "activeSubscriptions": 2, "outstandingAmount": 7814800,
"overdueAmount": 1200000, "paidToDate": 32000000 },
  "isActive": true
}
```

- `portalStatus` is computed: `none` (no login yet), `active`, `disabled`.
- `stats` can come from one aggregation per page of results. Don't run one query per row.
- A customer gets a login only by signing up on the customer site with the same email; sign-up links the login to this record (section 5). There is no invite flow.

# 9. Subscriptions endpoints

A subscription moves Draft → Quotation → Confirmed → Active → Closed, with Paused on the side. Each move is its own endpoint. The server returns `allowedActions` on every detail response, so the frontend shows exactly the buttons the server will accept.

**Only an Active subscription is billed; every arrow is one endpoint**




subscription statuses · 6 statuses, each arrow one action endpoint

The bottom line collects every way a deal ends before it starts: staff cancel, the customer rejects, or the quotation expires. Pause, renew and early close only work when the plan allows them.

| Method | Path | Who | Does |
|---|---|---|---|
| GET | `/subscriptions` | Admin, Staff, Finance | List. Filters: `status`, `customerId`, `planId`, `search` (number or customer name), `startFrom`, `startTo`, `expiringBefore` |
| POST | `/subscriptions` | Admin, Staff | Create as Draft (from a template or from scratch) |
| POST | `/subscriptions/preview` | Admin, Staff | Price a draft without saving (live totals in the quote builder) |
| GET | `/subscriptions/:id` | Admin, Staff, Finance | Full detail with priced lines, totals, allowedActions, history |
| PATCH | `/subscriptions/:id` | Admin, Staff | Edit (what's editable depends on status) |

| Method | Path | Who | Does |
|---|---|---|---|
| DELETE | `/subscriptions/:id` | Admin, Staff | Delete, Draft only |
| POST | `/subscriptions/:id/send` | Admin, Staff | Draft → Quotation |
| POST | `/subscriptions/:id/confirm` | Admin, Staff | Quotation → Confirmed |
| POST | `/subscriptions/:id/activate` | Admin, Staff | Confirmed → Active (manual; normally automatic) |
| POST | `/subscriptions/:id/pause` | Admin, Staff | Active → Paused |
| POST | `/subscriptions/:id/resume` | Admin, Staff | Paused → Active |
| POST | `/subscriptions/:id/renew` | Admin, Staff | Extend the expiry date |
| POST | `/subscriptions/:id/close` | Admin, Staff | Active or Paused → Closed |
| POST | `/subscriptions/:id/cancel` | Admin, Staff | Draft, Quotation or Confirmed → Closed |
| POST | `/subscriptions/:id/invoices` | Admin, Staff | Invoice the next period now ("Create invoice now") |

For a Finance user, `allowedActions` on a subscription is always empty.

## List

```
// GET /subscriptions?status=active&page=1&limit=20
{
  "data": [
    {
      "id": "6721f0c2a1b2c3d4e5f60760",
      "number": "SUB/2026/0042",
      "customer": { "id": "6721f0c2a1b2c3d4e5f60750", "name": "Acme Ltd" },
      "plan": { "id": "6721f0c2a1b2c3d4e5f60720", "name": "Pro Yearly",
"billingPeriod": "yearly" },
      "status": "active",
      "startDate": "2026-01-01", "expiryDate": "2026-12-31",
"nextInvoiceDate": "2027-01-01",
      "recurringTotal": 12814800
    }
  ],
  "meta": {
    "page": 1, "limit": 20, "total": 1248, "totalPages": 63,
    "statusCounts": { "draft": 8, "quotation": 21, "confirmed": 6, "active":
1248, "paused": 14, "closed": 115 }
  }
}
```

- `statusCounts` feeds the status tabs. It ignores the `status` filter but respects the others.
- `recurringTotal` = what one period costs today, tax included. Store it on the subscription whenever lines change, so lists don't price every row.

## Create

```
// POST /subscriptions, from a template
{
  "customerId": "6721f0c2a1b2c3d4e5f60750",
  "templateId": "6721f0c2a1b2c3d4e5f60770",
  "startDate": "2027-01-01"
}

// POST /subscriptions, from scratch
{
  "customerId": "6721f0c2a1b2c3d4e5f60750",
  "planId": "6721f0c2a1b2c3d4e5f60720",
  "startDate": "2027-01-01",
  "expiryDate": "2027-12-31",
  "paymentTermDays": 30,
```

```
  "salespersonId": "6721f0c2a1b2c3d4e5f60740",
  "notes": "",
  "lines": [
    { "productId": "6721f0c2a1b2c3d4e5f60710", "variantId":
"6721f0c2a1b2c3d4e5f60711", "quantity": 25 },
    { "productId": "6721f0c2a1b2c3d4e5f60730", "variantId": null, "quantity":
1, "unitPrice": 600000 }
  ]
}
```

- With `templateId`: copy the template's plan, lines and payment terms. Fields sent in the body win over the template's.
- Without lines: add one line for the plan's product at `minQuantity`.
- Assign `number` at creation. `status` = `draft`. `expiryDate` defaults to startDate + 1 year − 1 day.
- `unitPrice` is optional. Omitted = default price (section 14). Sent = staff override.
- Validate: plan available on `startDate` (`PLAN_NOT_AVAILABLE`); plan-product quantity ≥ `minQuantity` (`BELOW_MIN_QUANTITY`); `expiryDate` ≥ `startDate`; products and variants exist and are active.

`POST /subscriptions/preview` takes the same body and returns the `lines` and `totals` blocks shown below, without saving anything or using any number.

## Detail

```
// GET /subscriptions/:id
{
  "data": {
    "id": "6721f0c2a1b2c3d4e5f60760",
    "number": "SUB/2026/0042",
    "status": "active",
    "customer": { "id": "6721f0c2a1b2c3d4e5f60750", "name": "Acme Ltd",
"email": "billing@acme.com" },
    "plan": {
      "id": "6721f0c2a1b2c3d4e5f60720", "name": "Pro Yearly",
"billingPeriod": "yearly", "minQuantity": 10,
      "rules": { "autoClose": true, "closable": false, "pausable": false,
"renewable": true }
    },
    "template": { "id": "6721f0c2a1b2c3d4e5f60770", "name": "Pro Yearly –
Standard" },
    "salesperson": { "id": "6721f0c2a1b2c3d4e5f60740", "name": "Riya Sharma"
},
    "startDate": "2026-01-01",
```

```
    "expiryDate": "2026-12-31",
    "isExpired": false,
    "quotationValidUntil": "2025-12-29",
    "paymentTermDays": 30,
    "nextInvoiceDate": "2027-01-01",
    "lines": [
      {
        "id": "6721f0c2a1b2c3d4e5f60761",
        "productId": "6721f0c2a1b2c3d4e5f60710", "productName": "Brightloop
CRM",
        "variantId": "6721f0c2a1b2c3d4e5f60711", "variantLabel": "Brand:
Odoo",
        "quantity": 25, "unitPrice": 456000,
        "subtotal": 11400000,
        "discount": { "discountId": "6721f0c2a1b2c3d4e5f60780", "name": "Bulk
seats 10%", "amount": 1140000 },
        "orderDiscountShare": 0,
        "taxes": [ { "taxId": "6721f0c2a1b2c3d4e5f60790", "name": "GST 18%",
"type": "percentage", "rate": 18, "amount": 1846800 } ],
        "taxAmount": 1846800,
        "total": 12106800
      },
      {
        "id": "6721f0c2a1b2c3d4e5f60762",
        "productId": "6721f0c2a1b2c3d4e5f60730", "productName": "Analytics
add-on",
        "variantId": null, "variantLabel": null,
        "quantity": 1, "unitPrice": 600000,
        "subtotal": 600000, "discount": null, "orderDiscountShare": 0,
        "taxes": [ { "taxId": "6721f0c2a1b2c3d4e5f60790", "name": "GST 18%",
"type": "percentage", "rate": 18, "amount": 108000 } ],
        "taxAmount": 108000,
        "total": 708000
      }
    ],
    "orderDiscount": null,
    "totals": { "untaxedAmount": 12000000, "discountTotal": 1140000,
"taxTotal": 1954800, "total": 12814800 },
    "allowedActions": ["edit", "renew", "createInvoice"],
    "invoiceSummary": { "count": 11, "outstandingAmount": 7814800,
"lastInvoice": { "id": "6721f0c2a1b2c3d4e5f607a0", "number": "INV/2026/0310",
"status": "confirmed" } },
    "history": [
      { "at": "2026-01-02T00:30:00Z", "by": { "id": "system", "name":
"System" }, "action": "activate", "from": "confirmed", "to": "active",
"note": null },
```

```
      { "at": "2025-12-20T11:02:00Z", "by": { "id":
"6721f0c2a1b2c3d4e5f60701", "name": "Vikram Iyer" }, "action": "confirm",
"from": "quotation", "to": "confirmed", "note": "Accepted in portal" }
    ],
    "notes": "",
    "createdAt": "2025-12-18T10:00:00Z", "updatedAt": "2026-10-31T00:30:00Z"
  }
}
```


- Lines and totals are priced as if invoiced today. This is the same function the billing run uses, so the screen and the invoice always match.
- `allowedActions` possible values: `edit`, `delete`, `send`, `confirm`, `activate`, `pause`, `resume`, `renew`, `close`, `cancel`, `createInvoice`.
- `history` is newest first.

## What can be edited (PATCH)

| Status | Editable fields | Notes |
|---|---|---|
| Draft, Quotation | Everything: customer, plan, dates, payment terms, salesperson, notes, lines, unit prices | Editing a sent quotation is allowed and logged in history |
| Confirmed | Dates, salesperson, notes, lines | |
| Active, Paused | Lines (quantities, add or remove add-ons), salesperson, notes | Line changes apply from the next invoice. No proration in v1. |
| Closed | Nothing | `409 NOT_EDITABLE` |

Lines in PATCH: send the full array. A line with `id` updates that line; without `id` it is new; a missing line is removed. The plan's product line can't be removed and must stay ≥ `minQuantity`.

## Actions

Every action returns the updated subscription (same shape as Detail). Each writes a `history` entry. An action not allowed now returns `409 INVALID_TRANSITION`; one blocked by the plan returns `409 PLAN_RULE`.

| Action | From → To | Allowed when | Also does |
|---|---|---|---|
| `send` | Draft → Quotation | At least one line | Sets `quotationValidUntil` = today + template `validityDays` (14 if no template). Emails the customer if they have a login. |
| `confirm` | Quotation → Confirmed | Quotation not expired | If `startDate` ⩽ today, goes straight on to Active |
| `activate` | Confirmed → Active | Any time (demo button) | `nextInvoiceDate` = startDate |
| `pause` | Active → Paused | `rules.pausable` | Sets `pausedAt`. No invoices while paused. |
| `resume` | Paused → Active | Always | `nextInvoiceDate` = the later of its old value and today. Paused periods are never billed. |
| `renew` | Active or Paused (stays) | `rules.renewable` | `expiryDate` moves forward by one more term of the same length. Optional body `{ "newExpiryDate": "YYYY-MM-DD" }`. |
| `close` | Active or Paused → Closed | `rules.closable`, or already past `expiryDate` | `closeReason` = `closed_early` (or `expired`). `nextInvoiceDate` = null. Body `{ "note": "..." }`. |
| `cancel` | Draft, Quotation, Confirmed → Closed | Always | `closeReason` = `cancelled` |
| `createInvoice` | Active (stays) | Next period starts on or before `expiryDate` | Bills the next period now, exactly like the billing run. Returns `201` with the new invoice. |

### Automatic changes (nightly job)

The nightly job (section 10) does these first, in this order, writing `by: "system"` in history:

1. Quotation past `quotationValidUntil` → Closed ( `quotation_expired` ).
2. Confirmed with `startDate` ⩽ today → Active, `nextInvoiceDate` = startDate.
3. Active or Paused past `expiryDate` :
   - If `rules.autoClose` → Closed ( `expired` ).
   - Otherwise it stays, with `isExpired: true` . Billing stops after the expiry date until staff renew or close it.

## 10. Invoices and the billing run

Nobody types an invoice. The billing run creates Draft invoices from due subscriptions; staff confirm and send them; payments mark them Paid. One database index plus one transaction per period guarantee nothing is billed twice or skipped.

| Method | Path | Who | Does |
|---|---|---|---|
| GET | `/invoices` | Admin, Staff, Finance | List. Filters: `status` , `overdue=true` , `customerId` , `subscriptionId` , `search` (number or customer), `invoiceFrom` , `invoiceTo` , `dueFrom` , `dueTo` |
| GET | `/invoices/:id` | Admin, Staff, Finance | Full invoice with lines, payments, company details, allowedActions |
| PATCH | `/invoices/:id` | Admin, Staff | Draft only: `invoiceDate` , `dueDate` , `notes` |
| POST | `/invoices/:id/confirm` | Admin, Staff | Draft → Confirmed |
| POST | `/invoices/:id/cancel` | Admin, Staff | Draft or Confirmed → Cancelled |
| POST | `/invoices/:id/send` | Admin, Staff, Finance | Email it to the customer (also used as an overdue reminder) |
| POST | `/invoices/bulk/confirm` | Admin, Staff | Confirm many: `{ "ids": [...] }` |

| Method | Path | Who | Does |
|---|---|---|---|
| POST | `/invoices/bulk/send` | Admin, Staff, Finance | Send many: `{ "ids": [...] }` |
| POST | `/billing/run` | Admin, Staff (Finance: `dryRun: true` only) | Run billing now, or preview it with `dryRun` |
| GET | `/billing/last-run` | Admin, Staff, Finance | Result of the most recent run |

For a Finance user, an invoice's `allowedActions` holds at most `send` and `recordPayment`.

## List

```
// GET /invoices?status=confirmed&overdue=true
{
  "data": [
    {
      "id": "6721f0c2a1b2c3d4e5f607a1",
      "number": "INV/2026/0298",
      "customer": { "id": "6721f0c2a1b2c3d4e5f60750", "name": "Acme Ltd" },
      "subscription": { "id": "6721f0c2a1b2c3d4e5f60760", "number":
"SUB/2026/0042" },
      "invoiceDate": "2026-09-02", "dueDate": "2026-10-02",
      "total": 1200000, "amountPaid": 0, "balance": 1200000,
      "status": "confirmed", "isOverdue": true, "daysOverdue": 29
    }
  ],
  "meta": {
    "page": 1, "limit": 20, "total": 37, "totalPages": 2,
    "statusCounts": { "draft": 128, "confirmed": 64, "paid": 2031,
"cancelled": 9, "overdue": 37 }
  }
}
```

**Overdue** is not a stored status. It means: status `confirmed`, `balance` > 0 and `dueDate` before today (IST). Compute `isOverdue` and `daysOverdue` in responses; `overdue=true` filters on the same rule.

## Detail

Same line, discount, tax and total fields as the subscription detail (section 9), plus:

```
{
  "data": {
    "id": "6721f0c2a1b2c3d4e5f607a0",
    "number": "INV/2026/0310",
    "status": "confirmed",
    "subscription": { "id": "6721f0c2a1b2c3d4e5f60760", "number":
"SUB/2026/0042" },
    "customer": { "id": "6721f0c2a1b2c3d4e5f60750", "name": "Acme Ltd" },
    "customerSnapshot": { "name": "Acme Ltd", "email": "billing@acme.com",
"billingAddress": { "...": "..." }, "taxId": "29ABCDE1234F1Z5" },
    "company": { "name": "Brightloop Software", "address": "...", "taxId":
"..." },
    "periodStart": "2026-11-01", "periodEnd": "2026-11-30",
    "invoiceDate": "2026-10-31", "dueDate": "2026-11-30",
    "lines": [ "...same shape as subscription lines..." ],
    "orderDiscount": null,
    "untaxedAmount": 12000000, "discountTotal": 1140000, "taxTotal": 1954800,
"total": 12814800,
    "amountPaid": 5000000, "balance": 7814800,
    "isOverdue": false, "daysOverdue": 0,
    "payments": [
      { "id": "6721f0c2a1b2c3d4e5f607b0", "method": "bank_transfer",
"amount": 5000000, "paymentDate": "2026-11-05", "note": "UTR 88213",
"recordedBy": { "id": "6721f0c2a1b2c3d4e5f60740", "name": "Riya Sharma" } }
    ],
    "allowedActions": ["send", "recordPayment"],
    "sentAt": "2026-10-31T10:00:00Z", "sentCount": 1,
    "createdBy": "system"
  }
}
```


- **Print is done by the frontend** from this response (a print-friendly page). The backend doesn't need to make PDFs in v1.
- `allowedActions` possible values: `edit`, `confirm`, `cancel`, `send`, `recordPayment`.

## Invoice actions

| Action | From → To | Allowed when | Also does |
|---|---|---|---|
| `confirm` | Draft → Confirmed | Always | Lines and amounts are locked from now on. Sets `confirmedAt`. |

| Action | From → To | Allowed when | Also does |
|---|---|---|---|
| `cancel` | Draft or Confirmed → Cancelled | `amountPaid` = 0, else `409 HAS_PAYMENTS` | Sets `isVoid: true`, releases discount uses, sets `cancelledAt`. If it was the subscription's latest invoice and body `rebill` is true (default), moves `nextInvoiceDate` back to its `periodStart` so the next run bills that period again. |
| `send` | Confirmed or Paid (stays) | Customer has an email | Emails a summary + a customer-site link through Brevo (`sendInvoice`). Sets `sentAt`, `sentCount + 1`. A Draft returns `409 INVALID_TRANSITION`. |
| Paid | Confirmed → Paid | Automatic | Happens when payments bring `balance` to 0 (section 11) |

Bulk endpoints return per-item results and never fail as a whole:

```
{ "data": { "succeeded": ["6721f0c2a1b2c3d4e5f607a0"], "failed": [ { "id": "6721f0c2a1b2c3d4e5f607a2", "code": "INVALID_TRANSITION", "message": "Invoice is cancelled" } ] } }
```

## POST /billing/run

```
// request (both fields optional)
{ "asOfDate": "2026-10-31", "dryRun": true }

// 200
{
  "data": {
    "asOfDate": "2026-10-31",
    "dryRun": true,
    "lifecycle": { "quotationsExpired": 2, "activated": 3, "autoClosed": 1 },
    "dueSubscriptions": 131,
    "created": 128,
    "skippedAlreadyInvoiced": 3,
    "failed": [ { "subscriptionId": "6721f0c2a1b2c3d4e5f60799", "number":
"SUB/2026/0099", "code": "INTERNAL", "message": "..." } ],
    "invoiceIds": [],
    "startedAt": "2026-10-31T10:00:00Z", "finishedAt": "2026-10-31T10:00:04Z"
  }
}
```

- `asOfDate` defaults to today (IST). `dryRun: true` counts what would happen and writes nothing; the "Run billing" dialog calls it first.
- The same function runs every night through `node-cron` (`BILLING_CRON`, default `30 0 * * *`, time zone Asia/Kolkata). Only the instance with `RUN_JOBS=true` runs it.
- Save each run's result (a `billingRuns` collection is enough) for `GET /billing/last-run`.

## How the run works

1. Apply the automatic subscription changes (section 9).
2. Find due subscriptions: `status: "active"`, `nextInvoiceDate` ⩽ asOfDate, and `nextInvoiceDate` ⩽ `expiryDate`. Process them in batches of 100.
3. For each one, bill one period at a time, and repeat while it is still due (this catches up missed periods):
   a. `periodStart` = `nextInvoiceDate`. `periodEnd` = periodStart + 1 billing period − 1 day (date-fns `addDays` / `addWeeks` / `addMonths` / `addYears`).
   b. `invoiceDate` = asOfDate. `dueDate` = invoiceDate + `paymentTermDays`.
   c. Price the lines with the pricing engine (section 14), with discounts checked against `invoiceDate`.

   d. Inside one `session.withTransaction()`: reserve each applied discount's use, take the next invoice number, insert the Draft invoice, and move `nextInvoiceDate` to `periodEnd + 1 day` with a conditional update (`{ _id, nextInvoiceDate: periodStart }`).
   e. Duplicate-key error on `{ subscriptionId, periodStart }` = another run already billed it. The transaction rolls back (no number or discount use wasted); count it as `skippedAlreadyInvoiced`.
4. An error on one subscription goes into `failed` and never stops the run.

Why this holds: the unique index makes a second invoice for a period impossible. The invoice and the pointer move together, so a crash can't skip a period. The catch-up loop plus nightly and manual runs mean a missed night is billed on the next run.

## 11. Payments endpoints

A payment is recorded against one confirmed invoice, by any method, on any day. Partial payments are fine; the invoice turns Paid when its balance reaches 0.

| Method | Path | Who | Does |
|---|---|---|---|
| GET | `/payments` | Admin, Staff, Finance | List. Filters: `invoiceId`, `customerId`, `method`, `dateFrom`, `dateTo`, `search` (invoice number or customer) |
| POST | `/payments` | Admin, Staff, Finance | Record a payment |
| GET | `/payments/:id` | Admin, Staff, Finance | One payment |
| DELETE | `/payments/:id` | Admin | Reverse a mistaken payment |

```
// POST /payments
{
  "invoiceId": "6721f0c2a1b2c3d4e5f607a0",
  "method": "bank_transfer",
  "amount": 5000000,
  "paymentDate": "2026-11-05",
  "note": "Part payment, UTR 88213"
}

// 201: the payment plus the invoice's new state, so the screen updates
without another call
{
```

```
  "data": {
    "payment": {
      "id": "6721f0c2a1b2c3d4e5f607b0",
      "invoice": { "id": "6721f0c2a1b2c3d4e5f607a0", "number":
"INV/2026/0310" },
      "customer": { "id": "6721f0c2a1b2c3d4e5f60750", "name": "Acme Ltd" },
      "method": "bank_transfer", "amount": 5000000, "paymentDate": "2026-11-
05",
      "note": "Part payment, UTR 88213",
      "recordedBy": { "id": "6721f0c2a1b2c3d4e5f60745", "name": "Meera
(Finance)" },
      "createdAt": "2026-11-05T06:12:00Z"
    },
    "invoice": { "id": "6721f0c2a1b2c3d4e5f607a0", "number": "INV/2026/0310",
"status": "confirmed", "amountPaid": 5000000, "balance": 7814800 }
  }
}
```

List items use the `payment` shape above. `meta.totalAmount` = the sum of all payments matching the filters (not just the current page), for the "Total this period" line.

Rules:

- The invoice must be `confirmed`. Draft, Paid or Cancelled returns `409 INVALID_TRANSITION`.
- `amount` is an integer > 0 and ≤ the invoice `balance`, else `409 OVERPAYMENT`.
- `paymentDate` can't be in the future. `method` must be one of the enum values.
- **Use one transaction:** insert the payment, then update the invoice with a condition, `findOneAndUpdate({ _id, status: "confirmed", balance: { $gte: amount } }, { $inc: { amountPaid: amount, balance: -amount } })`. If nothing matched, abort and return `OVERPAYMENT` (this also stops two people recording the same payment at once). If the new balance is 0, set `status: "paid"` and `paidAt` in the same transaction.
- **DELETE** (admin only, body `{ "reason": "..." }`) reverses it in one transaction: `amountPaid` down, `balance` up, and a Paid invoice goes back to Confirmed with `paidAt: null`.

## 12. Dashboard and reports endpoints

The dashboard is the finance lead's home screen. It answers four questions in one place: how many subscriptions are active, how much came in, how much is owed, and what is overdue. Every report takes the same filters and must respond in under 2 seconds with the large seed data.

| Method | Path | Who | Does |
|---|---|---|---|
| GET | `/reports/summary` | Admin, Staff, Finance | The number tiles |
| GET | `/reports/revenue-by-month` | Admin, Staff, Finance | Bar chart: invoiced vs collected per month |
| GET | `/reports/subscriptions-by-plan` | Admin, Staff, Finance | Donut: active subscriptions and MRR per plan |
| GET | `/reports/aging` | Admin, Staff, Finance | Unpaid amounts grouped by days overdue |
| GET | `/reports/export` | Admin, Staff, Finance | CSV download of invoices, payments or subscriptions |

Shared filters: `from`, `to` (`YYYY-MM-DD`; default = current month), `planId`, `customerId`. The overdue table uses `GET /invoices?overdue=true&sort=dueDate`; recent payments use `GET /payments?sort=-paymentDate&limit=5`.

## GET /reports/summary

```
{
  "data": {
    "range": { "from": "2026-10-01", "to": "2026-10-31" },
    "activeSubscriptions": 1248,
    "newSubscriptions": 52,
    "closedSubscriptions": 9,
    "mrr": 96000000,
    "revenueCollected": 84000000,
    "invoicedAmount": 90000000,
    "outstandingAmount": 21000000,
    "overdueAmount": 6400000,
    "overdueCount": 37
  }
}
```

| Metric | Definition |
|---|---|
| `activeSubscriptions` | Status `active` right now. Ignores the date range. |
| `newSubscriptions` | `startDate` in range, and status `active`, `paused` or `closed` with reason `expired` / `closed_early` (so quotes that never started don't count) |

| Metric | Definition |
|---|---|
| `closedSubscriptions` | `closedAt` in range with reason `expired` or `closed_early` |
| `mrr` | Sum of `recurringNet` over active subscriptions, scaled to one month: daily × 365/12, weekly × 52/12, monthly × 1, yearly ÷ 12. Excludes tax. |
| `revenueCollected` | Sum of payment `amount` with `paymentDate` in range |
| `invoicedAmount` | Sum of invoice `total` for Confirmed and Paid invoices with `invoiceDate` in range |
| `outstandingAmount` | Sum of `balance` on Confirmed invoices right now |
| `overdueAmount`, `overdueCount` | Same, but only overdue invoices (section 10 rule) |

## GET /reports/revenue-by-month

`from` and `to` take months (`2026-01`), at most 24. Every month in the range appears, with zeros where nothing happened.

```
{ "data": [ { "month": "2026-09", "invoiced": 88000000, "collected": 79000000 }, { "month": "2026-10", "invoiced": 90000000, "collected": 84000000 } ] }
```

## GET /reports/subscriptions-by-plan

```
{ "data": [ { "plan": { "id": "6721f0c2a1b2c3d4e5f60720", "name": "Pro Yearly" }, "count": 236, "mrr": 41000000 } ] }
```

## GET /reports/aging

Buckets by days past due, for Confirmed invoices with a balance: `current` (not due yet), `1-30`, `31-60`, `61-90`, `90+`.

```
{ "data": [ { "bucket": "current", "count": 27, "amount": 14600000 }, { "bucket": "1-30", "count": 22, "amount": 3900000 }, { "bucket": "90+", "count": 3, "amount": 900000 } ] }
```

### GET /reports/export

`?type=invoices|payments|subscriptions` plus that list endpoint's filters. Returns `text/csv` with `Content-Disposition: attachment`. In the CSV, amounts are rupees with 2 decimals ( `45600.00` ) so spreadsheets read them. The frontend downloads it with the auth header, so it can't be a plain link.

### Keeping it under 2 seconds

- Each aggregation starts with a `$match` on indexed fields. Reports read the stored `balance` , `recurringTotal` and `recurringNet` ; they never re-price anything.
- Run the summary's separate queries in parallel ( `Promise.all` ).
- If a report is still slow, cache its result for 60 seconds, keyed by its filters.
- Prove it: time every report against the `--large` seed (section 15) before calling this done.

## 13. Customer portal endpoints

The React portal and the KMP mobile app both use these endpoints, and only these (plus `/auth` ). Every query is scoped to the logged-in user's `customerId` on the server; no `customerId` is ever sent by the client.

| Method | Path | Who | Does |
|---|---|---|---|
| GET | `/portal/summary` | Customer | Home cards |
| GET | `/portal/subscriptions` | Customer | Own subscriptions. Filter: `status` . Drafts are never shown. |
| GET | `/portal/subscriptions/:id` | Customer | One subscription with lines, totals, customer allowedActions |
| POST | `/portal/subscriptions/:id/accept` | Customer | Accept a quotation: Quotation → Confirmed |
| POST | `/portal/subscriptions/:id/reject` | Customer | Reject a quotation: Quotation → Closed ( `rejected` ) |
| POST | `/portal/subscriptions/:id/pause` | Customer | Behind a flag (below) |
| POST | `/portal/subscriptions/:id/resume` | Customer | Behind a flag |

| Method | Path | Who | Does |
|---|---|---|---|
| POST | `/portal/subscriptions/:id/renew` | Customer | Behind a flag |
| POST | `/portal/subscriptions/:id/close` | Customer | Behind a flag |
| GET | `/portal/invoices` | Customer | Own invoices. Filters: `status`, `unpaid=true`. Drafts are never shown. |
| GET | `/portal/invoices/:id` | Customer | One invoice with lines and payments |

```
// GET /portal/summary
{
  "data": {
    "activeSubscriptions": 1,
    "pendingQuotations": [ { "id": "6721f0c2a1b2c3d4e5f60763", "number":
"SUB/2026/0063", "plan": { "name": "Pro Monthly" }, "quotationValidUntil":
"2026-11-12", "recurringTotal": 1416000 } ],
    "outstandingAmount": 354000,
    "overdueCount": 1,
    "nextInvoice": { "subscriptionNumber": "SUB/2026/0051", "date": "2026-11-
10", "amount": 354000 }
  }
}
```

- Portal subscription and invoice responses use the same shapes as sections 9 and 10, minus staff-only fields: `history`, `salesperson`, `notes`, `createdBy`, `recordedBy`.
- Customer `allowedActions` values: `accept`, `reject` (Quotation, not expired), plus `pause`, `resume`, `renew`, `close` only when the flag is on and the plan rule allows it.
- Accept follows the same rules as staff `confirm` and writes history with the customer as the actor.
- **Flag:** `CUSTOMER_SELF_SERVICE=false` by default. When false, the four self-service routes return `403 FORBIDDEN` and never appear in `allowedActions`. This keeps the open decision (section 17) from blocking the build.
- Someone else's ID, or a draft: `404 NOT_FOUND`.
- Stretch, not v1: `POST /portal/invoices/:id/pay` for online payment.

# 14. Business rules the backend must enforce

One pricing function, `priceLines(subscription, onDate)`, is used everywhere: preview, subscription detail, billing run and "Create invoice now". If two places priced separately, the screen and the invoice would drift apart.

## Default unit price (set when a line is created)

- The plan's own product: `plan.price + variant.extraPrice`.
- Any other product (add-ons): `product.salesPrice + variant.extraPrice`.
- Staff may override `unitPrice` while the subscription is Draft or Quotation. The price is then stored on the line, so later catalog changes don't affect it.

## Pricing steps

1. **Line subtotal** = `unitPrice × quantity`.
2. **Line discount.** From discounts that are running on `onDate` (below) with `appliesTo.kind = "products"` and this product listed, keep those whose `minQuantity` ⩽ the line quantity and whose `minPurchaseAmount` ⩽ the subscription's total of line subtotals. Apply the one giving the biggest amount. Percentage: `round(subtotal × value / 100)`. Fixed: `min(value, subtotal)`.
3. **Subscription discount.** From running discounts with `appliesTo.kind = "subscription"`, keep those whose `minQuantity` ⩽ the total quantity and whose `minPurchaseAmount` ⩽ the sum of (subtotal − line discount). Apply the biggest. Split it across lines in proportion to their net amounts; the rounding remainder goes to the largest line. Each line's share is `orderDiscountShare`.
4. **Taxes per line**, on `subtotal − line discount − orderDiscountShare`. Matching taxes: active, and `appliesTo` is `all`, or lists the product's type, or lists the product. Percentage: `round(base × rate / 100)`. Fixed: `rate × quantity`. Several taxes each apply to the same base (no tax on tax).
5. **Totals:** `untaxedAmount` = sum of subtotals; `discountTotal` = line discounts + subscription discount; `taxTotal` = sum of line taxes; `total = untaxedAmount − discountTotal + taxTotal`.

Rounding: `Math.round` on paise, for each line and each tax, never on totals. At most one product discount per line and one subscription discount per invoice; discounts never stack.

**Running discount** on a date: `isActive`, `startDate` ⩽ date ⩽ `endDate`, and `usageLimit` is null or `usedCount < usageLimit`.

**Discount usage:** 1 use = 1 invoice it was applied to. The billing transaction reserves the use with a conditional update, `updateOne({ _id, $or: [{ usageLimit: null }, { $expr: { $lt: ["$usedCount", "$usageLimit"] } }] }, { $inc: { usedCount: 1 } })`. If that matches nothing, re-price without the discount. Cancelling the invoice gives the use back. Preview and detail screens never use one up.

### Test case 1: line discount (from the brief's "more seats" scene)

Pro Yearly, Brightloop CRM with brand Odoo (₹4,000 + ₹560), 25 seats. Discount "Bulk seats 10%" on CRM, minimum 20 seats. GST 18% on services and add-ons.

| Line | Subtotal (₹) | Discount (₹) | Taxable (₹) | GST 18% (₹) | Total (₹) |
|---|---|---|---|---|---|
| CRM (Odoo), 25 × 4,560 | 1,14,000 | −11,400 | 1,02,600 | 18,468 | 1,21,068 |
| Analytics add-on, 1 × 6,000 | 6,000 | 0 | 6,000 | 1,080 | 7,080 |
| **Invoice** | **1,20,000** | **−11,400** | **1,08,600** | **19,548** | **1,28,148** |

Automated test: `total` must equal `12814800` paise.

### Test case 2: subscription discount

Basic Monthly, CRM Generic, 1 × ₹3,000. Discount "Launch ₹500 off" on whole subscriptions, minimum purchase ₹2,000. Taxable = ₹3,000 − ₹500 = ₹2,500. GST = ₹450. Total = ₹2,950. Automated test: `total` must equal `295000` paise.

## Other rules

| Rule | Where |
|---|---|
| One account per email (stored lowercase); password rule | Section 5 |
| Sign-up only ever creates customers; only admin creates staff, finance and admin users | Sections 3, 7 |
| Plan must be available on the start date; plan-product quantity ≥ `minQuantity` | Section 9 |
| Status changes only through action endpoints. Reject `status` in any PATCH body. | Sections 9, 10 |
| Invoice lines and amounts never change after Confirm; invoices store a copy of names, prices and rates | Sections 4, 10 |

| Rule | Where |
|---|---|
| Never bill a period twice; never skip one | Section 10 |
| Payments never exceed the balance; payment + invoice update in one transaction | Section 11 |
| Customers never see drafts or another customer's data | Section 13 |
| Money inputs must be integers ( `45.6` gets `400 VALIDATION_ERROR` ) | Section 2 |
| "Today" always means today in Asia/Kolkata | Section 2 |
| Referenced records are archived, never hard-deleted | Section 2 |

## 15. DevOps expectations

Production runs on AWS in Mumbai ( `ap-south-1` ): the API on Elastic Beanstalk, the database on MongoDB Atlas, emails through Brevo, and CloudFront giving web and API one HTTPS address. Locally, one `docker compose up` starts MongoDB and the API. Backend owns all of it, including the one-time account setup.

### Repo layout

```
brightloop/
  apps/
    api/                      Express + Mongoose (backend)
      src/
      Procfile                web: node src/server.js
      .platform/nginx/conf.d/upload.conf
      package.json, package-lock.json
    web/                      React + Vite (frontend)
    mobile/                   Kotlin Multiplatform (frontend)
  docs/api/                   Bruno or Postman collection, kept in sync with
this doc
  docs/samples/               sample-users.csv, sample-users-invalid.csv
  docker-compose.yml
  .github/workflows/ci.yml, deploy-api.yml
```

Suggested API packages (backend's call): `express` , `mongoose` , `zod` , `bcrypt` , `jsonwebtoken` , `date-fns` , `node-cron` , `multer` , `csv-parse` , `helmet` , `cors` , `express-rate-limit` , `pino-http` . Call Brevo with Node 20's built-in `fetch` ; no SDK needed.

## Local development

```
# docker-compose.yml
services:
  mongo:
    image: mongo:7
    command: ["--replSet", "rs0", "--bind_ip_all"]
    ports: ["27017:27017"]
    volumes: ["mongo-data:/data/db"]
    healthcheck:   # starts the replica set on first boot, so transactions
work
      test: ["CMD", "mongosh", "--quiet", "--eval", "try { rs.status().ok }
catch (e) { rs.initiate({ _id: 'rs0', members: [{ _id: 0, host: 'mongo:27017'
}] }).ok }"]
      interval: 5s
      retries: 30
  api:
    build: ./apps/api
    env_file: ./apps/api/.env
    ports: ["4000:4000"]
    depends_on:
      mongo: { condition: service_healthy }
volumes:
  mongo-data:
```

- Locally `EMAIL_PROVIDER=console`: reset links print in the API log and come back as `devResetUrl`.
- Running the API outside Docker? Connect with `mongodb://localhost:27017/brightloop?directConnection=true`. Transactions still work.
- The Android emulator reaches the laptop at `http://10.0.2.2:4000`; a real phone needs the laptop's LAN IP or the deployed URL.

## AWS and account setup (one time)

1. **MongoDB Atlas.** Cluster on AWS `ap-south-1` (free M0 is enough; transactions work). A database user with `readWrite` on `brightloop`. Network access: the Elastic Beanstalk instance's Elastic IP only (step 2). Use `0.0.0.0/0` only briefly while debugging.
2. **Elastic Beanstalk.** Application `brightloop-api`, environment `brightloop-api-prod`:
   - Platform: Node.js 20 on 64-bit Amazon Linux 2023.
   - **Single instance** (t3.small). This gives a fixed Elastic IP for the Atlas and Brevo allow-lists, and guarantees exactly one copy of the nightly billing job.

- Health check path `/api/v1/health`, enhanced health on, logs streamed to CloudWatch (7-day retention).
- Every variable from the table below goes in Configuration → Environment properties. Secrets live only there, never in git.
- The app listens on `process.env.PORT` (EB sets 8080; its nginx forwards port 80 to it). Set `app.set("trust proxy", true)` so rate limits see real client IPs.

3. **nginx upload limit.** nginx rejects bodies over 1 MB with a `413` before Express sees them. Add `.platform/nginx/conf.d/upload.conf` containing `client_max_body_size 2M;` so CSV imports get through.
4. **HTTPS through CloudFront (no domain needed).** The EB instance serves plain HTTP, and browsers block an HTTPS web page from calling an HTTP API. Create one CloudFront distribution:
   - Default behavior → an S3 bucket holding the React build (frontend uploads it). Add a CloudFront Function that rewrites paths without a file extension to `/index.html`, so `/staff/login` deep links work. Don't use custom error pages for this: they would also replace the API's JSON 404s.
   - Behavior `/api/*` → the EB environment URL as origin (HTTP to the origin), managed policies `CachingDisabled` and `AllViewerExceptHostHeader`, all HTTP methods allowed.
   - Result: one address, `https://<id>.cloudfront.net`, for the web app, the API and the mobile app. Same origin, so production needs no CORS. A custom domain later = an ACM certificate (in `us-east-1`) on this distribution.
5. **Brevo.**
   - Verify the sender: ideally the domain (SPF + DKIM records), at minimum one sender email.
   - Create an API key (SMTP & API → API keys) and put it in `BREVO_API_KEY`.
   - Create a transactional template "Password reset" using `{{ params.name }}`, `{{ params.resetUrl }}` and `{{ params.expiresInMinutes }}`; put its ID in `BREVO_TEMPLATE_RESET_ID`.
   - If "Authorised IPs" is on in Brevo's security settings, add the EB Elastic IP, or every send fails with `401`.
   - The free plan's 300 emails a day is plenty for the hackathon.
6. **Deploy access.** An IAM role for GitHub Actions (OIDC, no long-lived keys) with the `AdministratorAccess-AWSElasticBeanstalk` managed policy, plus S3 and CloudFront invalidation rights for the web deploy.

## CI/CD (GitHub Actions)

- **`ci.yml`** on every push and pull request: `npm ci`, lint, tests (use `MongoMemoryReplSet` from `mongodb-memory-server` so transactions work in tests).
- **`deploy-api.yml`** on push to `main`, after CI passes:
  1. Zip `apps/api` (src, package files, `Procfile`, `.platform`). Leave out `node_modules` and `.env`; EB installs dependencies itself.
  2. Deploy with `einaregilsson/beanstalk-deploy` (or `eb deploy`), version label = the git commit SHA.
  3. Wait for environment health Green, then smoke-test `https://<id>.cloudfront.net/api/v1/health`.
- **Rollback:** redeploy the previous version label from the EB console.
- **Web deploy** (frontend builds it; backend gives the pipeline its permissions): upload the Vite build to S3, then invalidate CloudFront `/*`.
- **Seeding production:** run `npm run seed` once from a laptop with the production `MONGODB_URI`, or through a manual `workflow_dispatch` job. Never on every deploy.

## Environment variables (`apps/api/.env.example`)

| Variable | Local example | Production | Purpose |
|---|---|---|---|
| `NODE_ENV` | `development` | `production` | `development` turns on `devResetUrl` |
| `PORT` | `4000` | set by EB (8080) | API port |
| `MONGODB_URI` | `mongodb://mongo:27017/brightloop?replicaSet=rs0` | Atlas `mongodb+srv://...` | Database |
| `JWT_SECRET` | any long string | 32+ random characters | Signs tokens |
| `JWT_EXPIRES_IN` | `7d` | `7d` | Token lifetime |
| `CORS_ORIGINS` | `http://localhost:5173` | CloudFront URL | Allowed web origins |
| `WEB_URL` | `http://localhost:5173` | `https://<id>.cloudfront.net` | Base for reset links |
| `EMAIL_PROVIDER` | `console` | `brevo` | Where email goes |
| `BREVO_API_KEY` | empty | from Brevo | Brevo authentication |
| `BREVO_SENDER_EMAIL` | empty | verified sender | From address |
| `BREVO_SENDER_NAME` | `Brightloop` | `Brightloop` | From name |
| `BREVO_TEMPLATE_RESET_ID` | empty | template ID | Reset email template; empty = HTML built in code |
| `TZ` | `Asia/Kolkata` | `Asia/Kolkata` | Business time zone |
| `RUN_JOBS` | `true` | `true` | Runs the nightly job (single instance) |

| Variable | Local example | Production | Purpose |
|---|---|---|---|
| `BILLING_CRON` | `30 0 * * *` | `30 0 * * *` | Nightly run time |
| `CUSTOMER_SELF_SERVICE` | `false` | `false` | Portal pause, resume, renew, close (section 13) |
| `COMPANY_NAME`, `COMPANY_ADDRESS`, `COMPANY_TAX_ID` | Brightloop Software, … | same | Printed on invoices, returned by `/meta` |

## Seed data (`npm run seed` wipes and refills the database)

Seeded accounts skip the forced password change. Every password meets the password rule.

| Role | Login page | Email | Password |
|---|---|---|---|
| Admin | Staff console | `admin@brightloop.test` | `Admin@2026!` |
| Staff | Staff console | `riya@brightloop.test` | `Staff@2026!` |
| Finance | Staff console | `meera@brightloop.test` | `Finance@2026!` |
| Customer (Acme Ltd) | Customer site | `billing@acme.test` | `Customer@2026!` |
| Customer (Kite Stores) | Customer site | `asha@kite.test` | `Customer@2026!` |

Data:

- **Products:** Brightloop CRM (service, ₹4,000, variants Brand: Odoo +₹560 and Brand: Generic +₹0), Analytics add-on (₹6,000), Extra storage add-on (₹1,500), Onboarding kit (physical, ₹2,500).
- **Plans:** at least one per billing period (daily, weekly, monthly, yearly), with different rule combinations. Include Pro Yearly (₹4,000 per seat, minimum 10, auto-close, renewable) and Basic Monthly (₹3,000, pausable, closable, renewable).
- **Taxes:** GST 18% (services and add-ons), GST 12% (physical).
- **Discounts:** Bulk seats 10% and Launch ₹500 off (running), plus one each that is expired, exhausted and scheduled.
- **Templates:** Pro Yearly – Standard (14 days), Basic Monthly – Starter (7 days).
- **Customers:** about 50; the two above have logins.
- **Subscriptions** in every status, including a quotation expiring tomorrow, a paused one, and one past expiry without auto-close.
- **Invoices** over the last 6 months in every status: some overdue by 5, 29 and 75 days, some partly paid.

- Test cases 1 and 2 from section 14 exist as real subscriptions (Acme and Kite), so totals can be checked on screen.
- **Sample CSVs** in `docs/samples/`: `sample-users.csv` (5 valid rows, all three access levels) and `sample-users-invalid.csv` (one bad email, one bad access level, one duplicate) for the import demo.

`npm run seed -- --large` adds 5,000 subscriptions, about 60,000 invoices and 40,000 payments for the 2-second check.

### Security basics

- `helmet`, `trust proxy`, and rate limits: `/auth` 10 per minute per IP, forgot password 3 per hour per email, import 5 per minute per admin.
- bcrypt cost 10. `JWT_SECRET`, `MONGODB_URI` and `BREVO_API_KEY` only in EB environment properties.
- Validate every body and query with `zod`. Never log passwords, tokens, reset links or uploaded CSV contents.
- Atlas and Brevo allow-lists hold only the EB Elastic IP.
- Put a request ID in every log line and in 500 responses, so a frontend bug report can be traced in CloudWatch.

### Before the demo

- [ ] EB environment health is Green and `/api/v1/health` answers through the CloudFront URL
- [ ] A real forgot-password email arrives from Brevo, for both a customer and a staff account
- [ ] The CloudFront URL works from a phone (KMP app) and from a laptop on another network
- [ ] Production is seeded, and the sample CSVs are on the demo laptop
- [ ] CloudWatch shows last night's billing job finishing without errors

## 16. Delivery order and definition of done

Backend ships in seven milestones, in this order. Each one unblocks a set of screens; until then, frontend uses mocks for them. Deploy to the dev URL from milestone 2 onward.

| Milestone | Backend delivers | Frontend works on meanwhile |
|---|---|---|
| M0 Kickoff (first hour) | Repo, docker compose, `/health`, error handler, auth middleware with `requireRole`, empty Bruno collection. AWS, Atlas and Brevo accounts created. Settle section 17 together. | React scaffold with two route groups (customer site, staff console), mock data copied from this doc |
| M1 Auth and users | Customer sign-up and login, staff login, forced password change, forgot and reset through Brevo (console locally), users create / disable / reset, CSV import with `dryRun`, `/meta`, seed accounts | Both login pages, sign-up, forgot and reset pages, "Set your password" screen, users list and CSV import screen |
| M2 Catalog, config, customers | Products, plans, templates, taxes, discounts, customers. **First deploy: Elastic Beanstalk + Atlas + CloudFront live.** | Catalog, config and customer screens on the real API; KMP app pointed at the CloudFront URL |
| M3 Subscriptions | CRUD, preview, all actions, `allowedActions`, history, pricing engine. Test cases 1 and 2 pass. | Quote builder, subscription list and detail |
| M4 Billing and money | Billing run with `dryRun`, nightly job, invoice actions and bulk, payments with transactions, invoice and quotation emails through Brevo | Invoices, "Run billing" dialog, payments, print view |
| M5 Portal and reports | `/portal/*`, `/reports/*`, CSV export, `--large` seed and speed check | Customer portal (web), dashboard, KMP customer screens |
| M6 Demo ready | CI/CD deploying from `main`, "Before the demo" checklist (section 15) done, bug fixes | Polish; rehearse the demo script below twice, together |

## Definition of done (every endpoint)

- Request and response match the shapes in this doc, including field names and money as paise.
- Input is validated and failures return the documented error codes.
- Role checks match the Who column.

- It's in the Bruno or Postman collection with a working example.
- It works on the dev URL with seed data.

## Must-pass checks

Automated tests or a scripted run in the collection. Tick each when it passes.

- [ ] Sign-up with an existing email in different case (`ASHA@kite.test`) returns `409 EMAIL_TAKEN`
- [ ] Sign-up with `"role": "admin"` in the body still creates a customer
- [ ] A staff account on `/auth/customer/login`, and a customer account on `/auth/staff/login`, both get `401 INVALID_CREDENTIALS`
- [ ] Password `Short1!A` (8 characters) is rejected; `Kite@2026!` is accepted
- [ ] A user with a temporary password gets `403 PASSWORD_CHANGE_REQUIRED` on everything except `/auth/me` and `/auth/change-password`
- [ ] After a password change or reset, the old token gets `401`
- [ ] Staff forgot password sends a Brevo email whose link opens `/staff/reset-password`; the link works once, and not after 30 minutes
- [ ] Forgot password for an unknown email still returns `200` and sends nothing
- [ ] A CSV with one bad row: the preview lists the error, and the real import creates 0 users
- [ ] A CSV that repeats an email, or uses an email that already exists, flags that row
- [ ] A valid 200-row CSV imports in one go, inside nginx's timeout
- [ ] Staff calling `POST /discounts` gets `403`; finance calling `POST /subscriptions` gets `403`
- [ ] Customer A requesting customer B's invoice gets `404`
- [ ] Test case 1 total = `12814800`; test case 2 total = `295000`
- [ ] Running billing twice for the same date: the second run creates 0 invoices
- [ ] Two billing runs started at the same moment create no duplicate invoices
- [ ] A subscription 3 periods behind gets 3 invoices from one run, one per period
- [ ] A payment larger than the balance returns `409 OVERPAYMENT`; a payment equal to the balance marks the invoice Paid
- [ ] Cancelling a subscription's latest invoice makes the next run bill that period again
- [ ] A discount with `usageLimit: 1` lands on exactly one invoice in a run of many
- [ ] A customer never sees a draft invoice or a draft subscription
- [ ] Every report responds in under 2 seconds on the `--large` seed

### Demo script (the brief's month, end to end)

1. The admin logs in on the staff console and imports `sample-users-invalid.csv`; the preview catches the bad rows. They import `sample-users.csv` instead.
2. The new finance lead logs in with the temporary password and is made to set a new one.
3. A new customer signs up on the customer site (password rule visible), then uses forgot password and receives the Brevo email.
4. The admin adds a product with the Odoo variant (+₹560), a plan and a bulk-seats discount.
5. Staff build a quotation from a template for 25 seats and send it.
6. The customer accepts it in the KMP mobile app.
7. The subscription goes Active; staff run billing. The invoice shows the discount and GST without anyone typing them.
8. Staff confirm and send the invoice; running billing again creates nothing new.
9. The finance lead records a part payment, then the rest; the invoice turns Paid.
10. The finance lead opens the dashboard: active subscriptions, revenue, outstanding and overdue, with filters.

## 17. Open decisions for the first hour

The brief doesn't settle these 15 choices. This spec already follows the proposed default for each, so building can start now. Agree or change each one together, then set its status; a change means editing the sections it names.

| # | Question | Proposed default (what this spec assumes) | Affects | Status |
|---|---|---|---|---|
| 1 | What can customers do themselves in the portal? | Accept or reject quotations. Pause, resume, renew and close are built but off (`CUSTOMER_SELF_SERVICE=false`). | 13 | Proposed |
| 2 | Can a customer subscribe without a quotation? | No. Sales creates every subscription in v1. | 9, 13 | Proposed |
| 3 | Online payments? | No. Staff and finance record payments by hand. | 11, 13 | Proposed |

| # | Question | Proposed default (what this spec assumes) | Affects | Status |
|---|---|---|---|---|
| 4 | What does renew do? | Extends the same subscription by one more term of the same length; the number stays. | 9 | Proposed |
| 5 | Changing seats or add-ons on an active subscription | Allowed. Applies from the next invoice. No proration. | 9 | Proposed |
| 6 | What counts as one discount use? | One invoice the discount was applied to | 14 | Proposed |
| 7 | Can discounts stack? | No. Best product discount per line, plus best subscription discount per invoice. | 14 | Proposed |
| 8 | Should the billing run confirm invoices itself? | No. It creates drafts; staff confirm. | 10 | Proposed |
| 9 | Bill at the start or end of each period? | Start (in advance) | 10 | Proposed |
| 10 | Do the hackathon rules allow hosted services (AWS, Atlas, Brevo)? | Check the rules. Fallback: Docker MongoDB and console emails on the demo laptop. | 15 | Proposed |
| 11 | What can the finance role do? | Read everything, record payments, send invoices and reminders, all reports. No changes to subscriptions, catalog or invoice status. | 3 | Proposed |
| 12 | Currency | INR only. Multi-currency is out of scope. | 2 | Proposed |
| 13 | CSV import when some rows are bad | All or nothing: fix the file and upload again | 7 | Proposed |
| 14 | Should new staff get a welcome email with their temporary password? | No. The admin shares it directly; Brevo sends only password resets, quotations and invoices. | 7, 15 | Proposed |

| # | Question | Proposed default (what this spec assumes) | Affects | Status |
|---|---|---|---|---|
| 15 | Custom domain? | No. Use the CloudFront URL; add a domain and certificate only if time allows. | 15 | Proposed |